Microsoft 365 Tenant Decommission & Retirement (Post-Migration)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The situation . . . 4

The outcome . . . 4

The approach — safe and governed by design . . . 4

Why it is low-risk . . . 4

Solution Overview 5

What this is . . . 5

The retirement model . . . 5

How it works . . . 6

Reference design . . . 6

Benefits . . . 6

Limitations & boundaries . . . 7

Deployment model . . . 7

Requirements 7

R1. Signed-off migration completion (the go/no-go) . . . 7

R2. Microsoft Graph read access for the tooling (least privilege) . . . 8

R3. Operator credential handling (no secrets in code or repo) . . . 8

R4. Retention schedule + legal-hold / eDiscovery register (from legal/records) . . . 9

R5. Records / evidence store for exports and closure artifacts . . . 9

R6. Global Administrator (or scoped closure roles) on the Microsoft 365 tenant . . . 9

R7. DNS control and change window for the routed domains . . . 10

R8. Named pilot scope for the teardown . . . 10

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

Implementation 12

Phase 0 — Assess & Design (read-only) . . . 13

Phase 1 — Completeness verification (read-only) . . . 13

Phase 2 — Data disposition vs retention / legal hold . . . 14

Phase 3 — Licensing / subscription wind-down (dry-run → apply) . . . 14

Phase 4 — Entra app-registration + identity cleanup (dry-run → apply) . . . 15

Phase 5 — DNS / integration re-point-or-remove (dry-run → apply, under change control) 15
Phase 6 — Formal tenant closure (irreversible past the deletion window) 16
Testing & Validation 17
T1 — Migration completeness (Gate 1) 17
T2 — Data disposition classification (Gate 2) 17
T3 — License wind-down (Gate 3) 17
T4 — App & identity cleanup (Gate 4) 18
T5 — DNS / integration (Gate 5) 18
T6 — Pre-closure guardrail (Gate 6) 18
T7 — Closure evidence pack (Gate 6) 19
End-to-end pilot acceptance 19
Troubleshooting 19
Authentication & tooling 19
Phase 1 — completeness 20
Phase 2 — data disposition 20
Phase 3 — licensing 20
Phase 4 — identity & apps 20
Phase 5 — DNS & mail flow 21
Phase 6 — closure 21
Best Practices 21
Sequence and gates 22
Safe-by-default automation 22
Data & legal discipline 22
Identity & app hygiene 22
DNS & integration 23
Evidence & auditability 23
Vendor-accuracy discipline 23
Deliverables Summary 23
Documentation 23
The 5 Python tools 24
Appendix A — PowerShell Scripts 25

Executive Summary

The situation

The enterprise has moved its people to Google Workspace, Google Cloud Platform, and Gemini Enterprise. The migration project ended at "everyone is live on Google" — but the **old Microsoft 365 tenant is still running**. It is still licensed (duplicate cost), still holds mailboxes, files, and identities (risk and orphaned data), and is still trusted by lingering app integrations and DNS records. Left alone, that residual tenant bleeds spend for months and carries real security and compliance exposure; shut down carelessly, it loses records the organization may be legally required to keep, or breaks integrations still quietly depending on it.

The outcome

A **clean, auditable retirement** of the Microsoft 365 tenant: migration completeness proven with data, data disposition handled in line with retention and legal-hold obligations, licensing wound down to stop duplicate spend, Entra app registrations and orphaned identities cleaned up, DNS and integrations re-pointed to Google or removed, and the tenant **formally closed** — with evidence captured at every step. The cost stops, the attack surface shrinks, and the closure survives audit.

The approach — safe and governed by design

- **Pilot-first, gated, reversible.** The work runs as six phases —

Assess → Completeness → Disposition → Licensing → Identity → DNS → Closure — each with an explicit pass/fail gate and a documented rollback. Everything reversible (disable before delete, re-assign a license, restore a DNS zone) happens **before** the tenant enters the irreversible deletion window. A bounded pilot scope proves the whole procedure before it touches the full tenant.

- **Nothing torn down until completeness passes.** No disposition or closure step runs while any user

still depends on Microsoft 365 — the completeness gate is a hard stop.

- **Legal owns the records call; the runbook owns the method and the evidence.** Legal-hold and

retention determinations stay with the legal/records/compliance functions. This solution inventories the data, exports what must be preserved to a governed store, and hands the evidence over — it never disposes of data on its own authority.

- **Safe-by-default automation.** Five included Python tools read the Microsoft tenant through the

Microsoft Graph API and default to read-only or dry-run; any change requires an explicit `--apply`, applied to the pilot scope first. Verification uses least-privilege read access, and no credentials or keys ever live in code.

Why it is low-risk

Every phase leaves an evidence file; every production change (licensing, identity, DNS, closure) is gated, piloted, and reversible until the final deletion window; and the one truly irreversible action — closing the tenant — runs last, only after a pre-closure guardrail check confirms nothing is orphaned and every

obligation is met. The result is the duplicate cost and residual risk gone for good, with an audit trail that proves it was done cleanly.

Solution Overview

What this is

This solution is the **decommission and retirement runbook** for the *old* Microsoft 365 tenant after an enterprise (80k–150k+ users) has cut over to Google Workspace, Google Cloud Platform, and Gemini Enterprise. Migration projects almost always end at "everyone is live on Google" — and then the source Microsoft 365 / Entra tenant sits for months: still licensed, still holding mailboxes and Drive/OneDrive data, still trusted by app registrations and SSO integrations, still authoritative in DNS for lingering records. That residual tenant is three problems at once — **duplicate cost** (paying for both platforms), **risk** (orphaned identities, standing app secrets, unmanaged data), and **compliance exposure** (records you may be legally obliged to keep, or obliged to dispose of).

This runbook turns "someone should shut that down eventually" into a **clean, auditable exit**: prove the migration is actually complete, dispose of or export data in line with retention and legal-hold obligations, wind down licensing and Entra app registrations, re-point or remove every DNS record and integration that still pointed at Microsoft, and then **formally close the tenant** — with evidence captured at every gate.

The retirement model

The work is organized as six phases, executed pilot-first with pass/fail gates and rollback windows, following the house model **Assess** → **(Design** →**) Configure** → **Pilot** → **Validate** → **Expand**:

1. **Completeness verification.** Prove — with data, not assertion — that mail, calendar, contacts,

Drive/OneDrive/SharePoint, Teams/Chat, and shared mailboxes/resources reached Google. No teardown step runs until completeness passes its gate.

1. **Data disposition vs retention / legal hold.** Inventory what data still lives only in Microsoft

365, cross-reference the org's **retention schedule** and any **legal holds / eDiscovery** matters, export what must be preserved to the org's records store, and dispose of the rest. *Legal-hold and records-retention determinations are made by the org's legal/records/compliance functions — this solution provides the technical inventory, export method, and evidence.*

1. **Licensing / subscription wind-down.** Reclaim and reduce paid seats, cancel or let lapse

subscriptions once data disposition is complete, and stop recurring spend — staged so nothing is cancelled while it is still needed for export or hold.

1. **Entra app-registration + identity cleanup.** Inventory app registrations, enterprise apps

(service principals), expiring/expired credentials and secrets, stale/guest accounts, and orphaned objects; remove or disable what the Google move made obsolete, on a documented schedule.

1. **DNS / integration re-point-or-remove.** Map every DNS record (MX, SPF, DKIM, DMARC, CNAME, TXT,

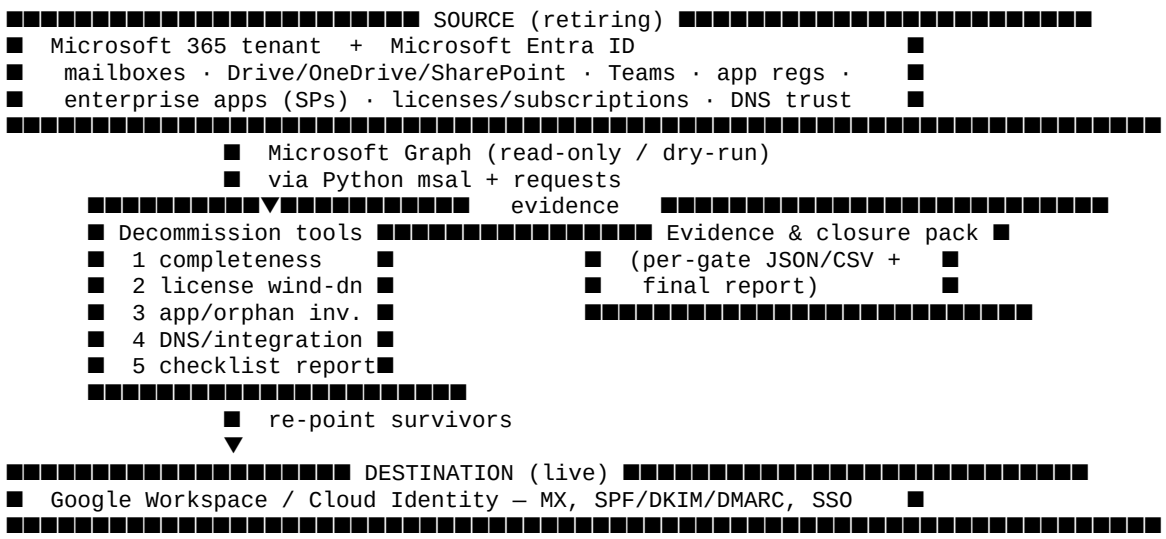
autodiscover, SRV) and every external integration still bound to the Microsoft tenant; re-point what must survive to Google and remove what is dead — under change control, because mail-flow records are high-impact.

1. **Formal tenant closure.** Once everything above has passed, execute the vendor closure path (subscription cancellation → deletion window) and capture the closure evidence pack.

How it works

Because the entire lifecycle runs on the *Microsoft* side of the migration, the source-of-truth for checks is the **Microsoft Graph API** and the Microsoft 365 / Entra admin centers. Per the Google category's automation rule, the five included tools are **Python** — they call **Microsoft Graph via `msal` (auth) and `requests` (calls)**, read-only or dry-run by default, so an engineer can *run to inventory and validate* rather than eyeball admin-center screens across a tenant of this size. The Google side of the destination (Workspace, Cloud Identity) is referenced only where a DNS record or integration must be **re-pointed to Google** — those steps cite the Google Admin console at exact detail.

Reference design



The Graph app registration created for the tooling is named `Decommission Read – Source-Tenant – v1` (application permissions, read-only scopes) and is itself removed in Phase 4 as the last object to go, after it has produced the closure evidence.

Benefits

• **Auditable exit.** Every gate produces evidence (JSON/CSV inventories, a signed checklist), so the closure survives audit and answers "prove nothing was orphaned and obligations were met".

• **Cost stops for good.** Licensing wind-down and tenant closure end duplicate spend that otherwise runs for months post-migration.

• **Risk reduced.** Standing app secrets, orphaned service principals, guest accounts, and stale trusts are enumerated and removed — shrinking the attack surface the migration left behind.

• **Compliance-safe.** Data disposition is gated on retention/legal-hold, and the technical evidence is handed to legal/records for their determination — the runbook never disposes of data unilaterally.

• **Safe by default.** All tooling is read-only or `--dry-run` until an operator explicitly passes `--apply`; production-affecting steps (DNS, closure) run under change control with rollback windows.

Limitations & boundaries

• **Legal determinations are deferred.** Whether a mailbox is under hold, which records must be kept and for how long, and whether disposition is permitted are **legal/records/compliance decisions**. This solution inventories and evidences; it does not decide.

• **Microsoft-side vendor detail is exact; preview features are flagged.** Graph endpoints, admin paths, and closure mechanics reflect current public Microsoft documentation. Anything version- or preview-dependent is flagged **"verify in your tenant"** rather than asserted.

• **Third-party integrations are handled at the pattern level.** Where an external app (ServiceNow, SAP, an SIEM, a backup vendor) trusts the Microsoft tenant, the runbook tells you how to *find* and *classify* the dependency; the vendor's own re-point/removal steps are deferred to their docs.

• **Not a migration tool.** This runs *after* cutover; it verifies completeness but does not move data. If completeness fails its gate, you return to the migration project — you do not proceed to teardown.

Deployment model

Assess → **(Design** →**) Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every production-affecting change. In this solution the "pilot" is a **bounded scope** — a single domain, a small set of mailboxes/accounts, or one DNS zone — proven end-to-end (inventory → disposition → cleanup → re-point) before the same procedure is expanded to the whole tenant. Closure itself is irreversible past the vendor's deletion window, so it is the last gate and requires all prior evidence signed off.

Requirements

These are the **solution-specific** prerequisites for decommissioning and retiring the source Microsoft 365 tenant after the move to Google. They build on the shared **Global Requirements — Google Workspace & Gemini Solutions** (`global-requirements-google.md`); in particular this solution relies on GR-2 (source **Microsoft 365 / Entra** admin access), GR-3 (**DNS control** for the routed domains), GR-5 (a **pilot scope** and change control), and GR-6 (**security, legal/eDiscovery, and data-governance stakeholders**). Do not re-verify the shared six here — verify the items below.

> *Reminder on the vendor boundary: Microsoft / Entra admin paths and Graph endpoints are authored at exact, current detail from Microsoft's public documentation. Where a capability or closure mechanic is version-dependent, it is flagged **"verify in your tenant"** — never invented.*

R1. Signed-off migration completion (the go/no-go)

• **Description:** A formal statement from the migration project that user-facing cutover to Google Workspace is **complete** — mail is delivered to Google, users work in Google, and no production workflow still depends on Microsoft 365.

• **Why required:** Teardown is destructive. Nothing in Phase 2 onward may run until the migration owner confirms cutover is done; the Phase 1 completeness verification *proves* this technically, but the business sign-off is the authority to begin.

• **How to verify:** Obtain the migration project's cutover sign-off (change record / email of record) and confirm the Google MX record is authoritative for each domain (`dig MX <domain>` returns Google, e.g. `smtp.google.com` / `aspmx.l.google.com`).

• **How to implement if missing:** Do **not** start. Return to the migration project and complete cutover; re-engage this runbook once sign-off exists.

R2. Microsoft Graph read access for the tooling (least privilege)

• **Description:** An **Entra app registration** for the decommission tools — named `Decommission Read – Source-Tenant – v1` — granted **application** permissions with **read-only** scopes and admin consent: `User.Read.All`, `Group.Read.All`, `Directory.Read.All`, `Application.Read.All`, `AuditLog.Read.All`, `Domain.Read.All`, `Organization.Read.All`, and (for mailbox/usage completeness) `Reports.Read.All`. No write scopes are needed for verification/inventory.

• **Why required:** The five Python tools authenticate via `msal` (client-credentials flow) and read the tenant through Graph. Read-only scopes keep the tooling safe by default.

• **How to verify:** In the **Microsoft Entra admin center → Identity → Applications → App registrations**, confirm the app exists, has the scopes above under** API permissions **with** "Granted for <tenant>"**, and that a** client secret or certificate** is available to the operator (stored in the environment, never in code — see R3).

• **How to implement if missing:** Register the app (**App registrations** → **New registration**), add the Graph **Application** permissions above, click **Grant admin consent**, then create a certificate (preferred) or client secret under **Certificates & secrets**. Record the **Application (client) ID**, **Directory (tenant) ID**, and credential in the operator's secret store.

R3. Operator credential handling (no secrets in code or repo)

• **Description:** The tenant ID, client ID, and client secret/certificate for R2 are provided to the tools **through the environment** (environment variables or the operator's secrets manager), never committed, never printed.

• **Why required:** These credentials can read the entire directory; leaking them is a breach. The tools are written to read `M365DECOM_TENANT_ID`, `M365DECOM_CLIENT_ID`, and `M365DECOM_CLIENT_SECRET` from the environment and refuse to run if absent — they contain no hardcoded credentials.

• **How to verify:** Confirm the three variables are set in the operator's session (`echo $M365DECOM_TENANT_ID` on the operator host) and that no credential appears in any script, config file, or the repo.

• **How to implement if missing:** Export the variables from the secret store at runtime (e.g. `export M365DECOM_CLIENT_SECRET=$(vault kv get -field=secret …)`); never paste secrets into files.

R4. Retention schedule + legal-hold / eDiscovery register (from legal/records)

• **Description:** The organization's **records-retention schedule** and the current **legal-hold / eDiscovery** register (which mailboxes, sites, or users are under hold, and any active matters), supplied by the legal/records/compliance functions (GR-6).

• **Why required:** Data disposition (Phase 2) is gated on this. Data under hold or within a retention window must be **exported/preserved**, not deleted. This solution provides the technical inventory and export method; **the determination of what to keep, and for how long, belongs to legal/records.**

• **How to verify:** Confirm you have the retention schedule and a current hold register, and a named approver who signs off disposition. In **Microsoft Purview** → **Data Lifecycle Management / eDiscovery** confirm which holds are active *(exact Purview navigation is version-dependent — verify in your tenant)*.

• **How to implement if missing:** Do not dispose of any data. Engage legal/records to produce the schedule and hold register and to designate the disposition approver before Phase 2.

R5. Records / evidence store for exports and closure artifacts

• **Description:** A destination the organization controls (e.g. a governed Google Drive shared drive, an archive bucket, or the records system) for **exported data** that must be preserved and for the **evidence pack** every phase produces.

• **Why required:** Preserved data must land somewhere durable and access-controlled; the audit trail (per-gate inventories, the final checklist/report) must be retained independently of the tenant being closed.

• **How to verify:** Confirm the store exists, is access-controlled, and its retention is at least as long as the longest applicable obligation from R4.

• **How to implement if missing:** Stand up (or nominate) the store and confirm its permissions before exporting anything.

R6. Global Administrator (or scoped closure roles) on the Microsoft 365 tenant

• **Description:** For the *write* actions in Phases 3–6 (license reclamation, app-registration

removal, DNS changes on Microsoft-hosted zones, subscription cancellation, tenant deletion), the operator needs **Global Administrator**, or the least-privilege combination the step names (e.g. **License Administrator**, **Application Administrator**, **Domain Name Administrator**, and **Billing Administrator**). *Tenant deletion prerequisites are version-dependent — verify current requirements in your tenant.*

• **Why required:** Verification is read-only (R2), but the teardown actions change or remove tenant objects and ultimately delete the tenant.

• **How to verify:** In **Entra admin center** → **Roles and administrators**, confirm the operator holds the required role(s).

• **How to implement if missing:** Have the tenant owner assign the role(s), preferably just-in-time for the change window, and remove them afterward.

R7. DNS control and change window for the routed domains

• **Description:** Authoritative control of each domain's DNS zone (GR-3) **and** an agreed change-control window, because MX/SPF/DKIM/DMARC/autodiscover changes are high-impact.

• **Why required:** Phase 5 re-points survivors to Google and removes dead Microsoft records; a mistake here breaks mail flow.

• **How to verify:** Confirm write access to each authoritative zone and a booked change window with rollback approval.

• **How to implement if missing:** Obtain zone access and schedule the change window with the DNS owners before Phase 5.

R8. Named pilot scope for the teardown

• **Description:** A **bounded first scope** (one domain, a small mailbox/account set, or a single DNS zone) to run the full procedure against before expanding tenant-wide — the GR-5 pilot applied to decommission.

• **Why required:** The house model is pilot-first; proving inventory → disposition → cleanup → re-point on a small scope de-risks the fleet-wide run.

• **How to verify:** Confirm the pilot scope is named and agreed with stakeholders.

• **How to implement if missing:** Choose and document the pilot scope before Phase 1's expand step.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Execute the decommission in the six phases below, following **Assess** → **(Design** →**) Configure** → **Pilot** → **Validate** → **Expand**. Every phase has a **pass/fail gate** and, where it changes production, a **rollback**

window. Run each phase against the **pilot scope** (R8) first, validate, then expand to the whole tenant. **Do not start Phase 2 until Phase 1 passes and R1 (migration sign-off) exists.**

Artifacts are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in later phases, testing, and the scripts. Establish these up front:

Artifact	Name
Graph read app (tooling)	`Decommission Read – Source-Tenant – v1`
Evidence/closure store (R5)	`Decommission Evidence – Source-Tenant – v1`
Pilot scope (R8)	`Decommission Pilot – <domain/OU> – v1`
Change record for DNS/closure	`Decommission Change – Prod – v1`

Set the operator environment once (R3), used by every tool:

```
export M365DECOM_TENANT_ID="<Directory (tenant) ID>"
export M365DECOM_CLIENT_ID="<Application (client) ID>"
export M365DECOM_CLIENT_SECRET="<from secret store — never committed>"
python -m pip install msal requests            # standard-library-first otherwise
```

Phase 0 — Assess & Design (read-only)

1. Confirm **R1 migration sign-off** and that Google MX is authoritative for every domain

(`dig MX <domain>`). If any domain still routes to Microsoft, that domain is **not** ready — remove it from this run.

1. Stand up `Decommission Read – Source-Tenant – v1` (R2) and confirm read-only consent.
2. Nominate `Decommission Evidence – Source-Tenant – v1` (R5) and the

`Decommission Pilot – <domain/OU> – v1` scope (R8).

1. Obtain the **retention schedule + legal-hold register** (R4) and the named disposition approver.

Gate 0 (design): sign-off exists, tooling reads the tenant, evidence store and pilot scope named, legal inputs in hand. *Fail →* resolve the missing input before proceeding.

Phase 1 — Completeness verification (read-only)

Goal: prove the migration actually landed before anything is torn down.

1. Run the completeness verifier against the pilot scope:

```
python scripts/verify_migration_completeness.py \
    --scope-file pilot-users.txt \
    --out evidence/phase1-completeness-pilot.json
```

It reads, per user, mailbox item counts and last-activity, OneDrive usage, and group/shared-mailbox coverage via Graph **usage reports** and directory reads, and compares against an expected baseline you supply (the migration project's source counts).

1. Review the report. Investigate any user with **large residual mailbox/OneDrive size** or

recent Microsoft-side activity (a signal they are still using M365, not Google).

1. **Expand:** re-run with the full user list (`--scope-file all-users.txt`).

Gate 1 (completeness): every in-scope user reconciles to the expected baseline within the agreed tolerance, and no user shows recent Microsoft-side activity. *Fail →* return the exceptions to the migration project; **do not proceed to teardown**. **Rollback:** none needed (read-only).

Phase 2 — Data disposition vs retention / legal hold

Goal: preserve what must be kept; dispose of the rest — on legal/records' authority.

1. Produce the disposition inventory (read-only) — mailboxes, OneDrive, and sites that still hold

data, joined to the R4 hold register:

```
python scripts/inventory_app_and_orphans.py --mode data --out evidence/phase2-data-inventory.json
```

*(The inventory tool covers identity/app objects **and** data-bearing objects; `--mode data` scopes it to disposition inputs.)*

1. Hand the inventory to **legal/records**. They mark each item **HOLD / RETAIN-UNTIL-<date> /

DISPOSE **against the retention schedule.** This solution does not make that call.**

1. For **HOLD / RETAIN** items: export to `Decommission Evidence – Source-Tenant – v1` (R5) using

the org's approved export path — Microsoft Purview eDiscovery export for held content, or the records team's tooling. *(Exact Purview export navigation is version-dependent — verify in your tenant.)* Record each export (item, size, hash, destination) in the evidence pack.

1. For **DISPOSE** items: only after the disposition approver signs the inventory, allow the tenant's

normal deletion to proceed (or apply the retention-driven deletion the records team directs).

Gate 2 (disposition): every data item is classified; all HOLD/RETAIN items are exported and evidenced; DISPOSE items are approved in writing. *Fail →* stop; unresolved-classification data is never deleted. **Rollback:** exports are additive and safe; do not action any DISPOSE until sign-off, so there is nothing to roll back before that point.

Phase 3 — Licensing / subscription wind-down (dry-run → apply)

Goal: stop duplicate spend, safely, without cutting off anything still needed for hold/export.

1. Model the wind-down (dry-run, the default):

```
python scripts/license_winddown_helper.py --out evidence/phase3-license-plan.json
```

It reports assigned SKUs per user, seats reclaimable now (users fully migrated and not on hold), seats to **retain** (accounts still needed for export/hold from Phase 2), and the projected monthly saving. **Nothing is changed in dry-run.**

1. Review the plan with the billing owner. Confirm no reclaimable account is on the R4 hold register.

2. Apply reclamation for the pilot scope:

```
python scripts/license_winddown_helper.py --apply --scope-file pilot-users.txt
```

The tool removes license assignments (does **not** cancel subscriptions) for the named users.

1. **Expand** to the full set once the pilot reclaim is validated. Cancel or let lapse the underlying

subscriptions only after Phase 2 export is complete and no account still needs a seat — subscription cancellation is done in the admin center under change control, not by the tool.

Gate 3 (licensing): reclaimable seats reclaimed, retained seats documented with reason, no held account de-licensed, projected saving recorded. *Fail →* re-run dry-run and correct the scope. **Rollback:** re-assign a license within the change window (assignment is reversible while the subscription is still active) if an account is found to still be needed.

Phase 4 — Entra app-registration + identity cleanup (dry-run → apply)

Goal: remove the standing app trust and orphaned identities the migration left behind.

1. Inventory app registrations, enterprise apps (service principals), credentials, and orphaned/stale

objects (read-only):

```
python scripts/inventory_app_and_orphans.py --mode identity \
    --out evidence/phase4-identity-inventory.json
```

It flags: app registrations with **expired or long-lived secrets/certs**, **service principals** with no recent sign-in, **guest** accounts, disabled/never-signed-in accounts, and apps whose reply URLs/SSO clearly target the retired estate.

1. Classify each object with the app/identity owners: **KEEP (migrated to Google) / DISABLE / REMOVE**.

Anything ambiguous is **disabled first**, not deleted, so it can be restored within the window.

1. Apply, pilot scope first, on a documented schedule:

```
python scripts/inventory_app_and_orphans.py --mode identity --apply \
    --action disable --scope-file pilot-apps.txt
```

Disable in the pilot, observe for the rollback window, then remove. **Expand** to the full tenant.

1. The tooling's own app, `Decommission Read – Source-Tenant – v1`, is the **last object removed**

— only after it has produced the Phase 6 closure evidence.

Gate 4 (identity): every app/identity object is classified and actioned; nothing ambiguous is deleted (only disabled); the schedule and approvals are evidenced. *Fail →* revert to disabled and re-classify. **Rollback:** re-enable a disabled object within the window (deletion is deferred until the disable soak passes).

Phase 5 — DNS / integration re-point-or-remove (dry-run → apply, under change control)

Goal: re-point survivors to Google, remove dead Microsoft records — without breaking mail.

1. Map the dependencies (read-only):

```
python scripts/map_dns_integration.py --domains-file domains.txt \
    --out evidence/phase5-dns-map.json
```

For each domain it resolves MX, SPF (TXT), DKIM (selector CNAMEs), DMARC, autodiscover, `enterprise registration`/`enterpriseenrollment` CNAMEs, Lync/Teams SRV, and the `MS=`/`msoid` verification records, and classifies each as **RE-POINT-TO-GOOGLE / REMOVE / KEEP**. It also lists Entra **verified domains** and app **reply URLs** as integration signals.

1. Confirm the target Google records for anything re-pointed (Google MX

`smtp.google.com` / `aspmx.l.google.com`; Google DKIM selector `google._domainkey`; SPF `include:_spf.google.com`) in the **Google Admin console** → **Apps** → **Google Workspace** → **Gmail** → **Authenticate email / Setup**.

1. Under change record `Decommission Change – Prod – v1`, apply to the **pilot domain first**:

remove Microsoft-only records (autodiscover, `enterpriseregistration`, Teams SRV, `MS=` verification once the domain is removed from the tenant), correct SPF to drop `include:spf.protection.outlook.com`, and confirm DMARC still passes. Watch mail flow through the rollback window.

1. **Expand** to remaining domains one zone at a time. The tool itself only *plans*; DNS edits are made

in the authoritative zone (or via its API) under change control — never blind.

Gate 5 (DNS/integration): for each domain, survivors point at Google, dead Microsoft records are gone, SPF/DKIM/DMARC still pass (`dig`/MXToolbox verification), and no integration is left pointing at the retired tenant. *Fail →* roll back the zone change. **Rollback:** restore the prior zone snapshot (take one before every change) within the change window.

Phase 6 — Formal tenant closure (irreversible past the deletion window)

Goal: close the tenant and capture the evidence pack.

1. **Pre-closure guardrail check** (read-only) — confirm nothing blocks a clean closure:

```
python scripts/decommission_report.py --precheck \
    --evidence-dir evidence/ --out evidence/phase6-precheck.json
```

It verifies all prior gates are evidenced, no domain is still verified-and-routing, no license is still assigned to a non-retained account, no app registration remains except the tooling app, and no unresolved HOLD item exists.

1. Remove each **verified domain** from the tenant (**Entra admin center** → **Settings** → **Domain names** →

remove), which requires the domain's records to be cleared first (Phase 5). *(Domain-removal prerequisites are version-dependent — verify in your tenant.)*

1. **Cancel the subscriptions** (Microsoft 365 admin center → **Billing** → **Your products** → cancel),

which starts Microsoft's **data-retention grace period** before deletion. Record the dates.

1. Generate the **closure evidence pack**:

```
python scripts/decommission_report.py --evidence-dir evidence/ \
```

```
--out evidence/M365DECOM-closure-report.html
```

1. Remove the tooling app `Decommission Read – Source-Tenant – v1` as the final object.
2. Allow the tenant to progress to deletion per Microsoft's process. *(Whether/where a "delete tenant" action is exposed vs. lapse-after-cancellation is version-dependent — verify current tenant-deletion requirements in your tenant; some tenants require all subscriptions removed and licenses/users cleared first.)*

Gate 6 (closure): precheck all-green, subscriptions cancelled, closure report generated and stored in `Decommission Evidence – Source-Tenant – v1`, tooling app removed. *Fail →* precheck names the blocker; resolve it before cancelling. **Rollback:** reversible only within Microsoft's grace period — re-activate a cancelled subscription before the deletion date if a blocker is found; past the deletion window closure is final.

Testing & Validation

Each phase gate is validated with the procedure below. Run every test against the **pilot scope** first, then re-run at tenant scope during **Expand**. Record each result in `Decommission Evidence – Source-Tenant – v1`. A phase is not "done" until its test **passes**.

> *All read-only tests can be run repeatedly and safely. Tests that confirm a *write* (license removed, app disabled, DNS changed) are run **after** the change, within the rollback window.*

T1 — Migration completeness (Gate 1)

- **Procedure:** Run `verify_migration_completeness.py` at full scope; open the JSON report.
- **Expected PASS:** Every in-scope user reconciles to the expected baseline within tolerance; residual mailbox/OneDrive sizes are at/under the agreed threshold; **no** user shows Microsoft-side activity after the cutover date.
- **FAIL signal:** Any user with an unexplained large residual or recent Microsoft-side sign-in/mail activity → that user is not fully migrated. Return to the migration project; do not proceed.

T2 — Data disposition classification (Gate 2)

- **Procedure:** Open `phase2-data-inventory.json`; confirm every item carries a legal/records classification (HOLD / RETAIN-UNTIL-<date> / DISPOSE) and that HOLD/RETAIN items have a recorded export (destination + hash) in the evidence pack.
- **Expected PASS:** 100% of items classified; every HOLD/RETAIN item exported and evidenced; every DISPOSE item carries the approver's sign-off.
- **FAIL signal:** Any unclassified item, or a HOLD item with no export record → stop; do not dispose.

T3 — License wind-down (Gate 3)

- **Procedure:** Run `license_winddown_helper.py` (dry-run) after applying reclamation; the "seats

still assigned" figure should equal only the **retained** (documented) accounts.

- **Expected PASS:** Reclaimable seats show as reclaimed; retained seats match the documented list with

reasons; **no** account on the R4 hold register is de-licensed; projected saving recorded.

- **FAIL signal:** A held/needed account shows de-licensed, or reclaimable seats remain assigned →

re-assign (rollback) or re-run apply, and correct the scope.

T4 — App & identity cleanup (Gate 4)

- **Procedure:** Run `inventory_app_and_orphans.py --mode identity` (read-only) after the disable/remove

actions.

- **Expected PASS:** No unexpected app registration remains except `Decommission Read – Source-Tenant

– v1`; no service principal with the retired estate's reply URLs remains enabled; flagged guest/stale accounts are disabled or removed per the schedule; nothing ambiguous was deleted (only disabled).

- **FAIL signal:** A standing secret/cert on a still-enabled app, or an orphaned SP still enabled →

disable and re-classify.

T5 — DNS / integration (Gate 5)

- **Procedure:** For each domain, verify DNS externally, independent of the tooling:

```
dig MX <domain> +short          # expect Google (aspmx.l.google.com / smtp.google.com)
dig TXT <domain> +short         # SPF should include _spf.google.com, NOT spf.protection.outlook.com
dig TXT _dmarc.<domain> +short  # DMARC record present and policy intact
dig CNAME google._domainkey.<domain> +short   # Google DKIM selector resolves
dig CNAME autodiscover.<domain> +short        # Microsoft autodiscover REMOVED (NXDOMAIN)
```

Send a test message to a user on the domain and confirm delivery to Google.

- **Expected PASS:** MX → Google; SPF/DKIM/DMARC pass and reference Google, not Microsoft; Microsoft-only

records (autodiscover, `enterpriseregistration`, Teams SRV, stale `MS=` verification) return NXDOMAIN; test mail delivers to Google; no integration still resolves to the retired tenant.

- **FAIL signal:** Mail bounces/misroutes, SPF/DMARC fails, or a Microsoft record still resolves →

restore the pre-change zone snapshot (rollback) and re-plan.

T6 — Pre-closure guardrail (Gate 6)

- **Procedure:** Run `decommission_report.py --precheck` before cancelling any subscription.

- **Expected PASS (all-green):** all prior gates evidenced; no domain still verified-and-routing; no

license assigned to a non-retained account; no app registration except the tooling app; zero unresolved HOLD items.

- **FAIL signal:** Any red check → resolve the named blocker before cancelling.

T7 — Closure evidence pack (Gate 6)

- **Procedure:** Generate `M365DECOM-closure-report.html`; confirm it is stored in the evidence store

and contains every phase's gate result, the disposition sign-offs, the license saving, the identity actions, the DNS end-state, and the subscription-cancellation dates.

- **Expected PASS:** Report generated, complete, and archived; subscriptions show cancelled with the

deletion date recorded; tooling app removed as the final step.

- **FAIL signal:** Missing evidence, or a subscription still active → do not declare closure complete.

End-to-end pilot acceptance

Before expanding tenant-wide, the pilot scope must pass **T1–T5** end to end (completeness → disposition → license → identity → DNS) with evidence, and a rollback must have been demonstrated at least once (e.g. re-enable a disabled app, restore a zone snapshot) to prove the safety net works.

Troubleshooting

Common failure modes across the six phases, with resolutions. Symptoms are grouped by where they appear. All tooling is read-only or dry-run by default, so most issues here are diagnostic, not data-loss — the exceptions (DNS, closure) are called out.

Authentication & tooling

`ERROR: Set M365DECOM_TENANT_ID / … in the environment` The tools read credentials only from the environment (R3). Export `M365DECOM_TENANT_ID`, `M365DECOM_CLIENT_ID`, and `M365DECOM_CLIENT_SECRET` from your secret store in the same shell, then re-run. Never paste them into a file.

`Token request failed: … invalid_client` **/** `AADSTS7000215` The client secret is wrong or expired. Regenerate the secret/cert for `Decommission Read – Source-Tenant – v1` in **Entra** → **App registrations** → **Certificates & secrets**, update the environment variable, and re-run. If you rotated to a certificate, confirm the tool's `msal` app is configured for it.

`Graph GET … -> 403` **(Authorization_RequestDenied)** The app is missing a scope or admin consent. In **App registrations** → **API permissions**, confirm the read scopes from R2 are present **and** show "Granted for <tenant>". Application permissions require admin consent — re-consent if the tick is missing.

`msal not installed` **/** `--graph needs: pip install msal requests` Install the client libraries: `python -m pip install msal requests` (and `dnspython` for cleaner DNS resolution in `map_dns_integration.py`).

Runs are very slow / `429` **retries in stderr.** Graph is throttling a large tenant. The tools back off automatically on 429/5xx. For 100k+ users, scope the run (`--scope-file`) and run in batches rather than the whole tenant at once; the reports are additive.

Phase 1 — completeness

A user shows `FAIL` for a post-cutover Microsoft-side sign-in. That user is still using Microsoft 365 — completeness genuinely fails. Do not proceed to teardown for that scope. Return the user to the migration project; re-run after they are confirmed on Google.

Large OneDrive residual flagged but the user is definitely migrated. Common causes: version history, recycle-bin contents, or a shared-library sync not part of the personal migration scope. Confirm with the migration project whether the residual is in-scope; if it is expected and dispositioned, record it in the evidence and lower isn't required — adjust `--residual-mb` only with sign-off, never to mask a real gap.

`ABSENT — user not found in directory`**.** The account was already removed upstream. This is noted, not failed. Confirm it was intentional and that any data it held was dispositioned in Phase 2 before it disappeared.

Phase 2 — data disposition

Inventory items remain `UNCLASSIFIED`. Legal/records has not yet marked them HOLD / RETAIN / DISPOSE. **Do not dispose of anything unclassified** — the pre-closure guardrail (`decommission_report.py --precheck`) will FAIL while any UNCLASSIFIED item remains. Escalate to the disposition approver.

Held content cannot be exported. Purview eDiscovery export navigation and prerequisites are version-dependent — verify in your tenant. If export stalls, confirm the operator has the eDiscovery role and that the hold is correctly scoped; engage the records team's tooling if Purview export is unavailable in your edition.

Phase 3 — licensing

A held/needed account appears in `reclaimable`. It is missing from the hold/retain lists. Add its UPN to `--hold-file` (legal hold) or `--retain-file` (still needed) and re-run the dry-run. Never `--apply` until the reclaimable list is clean.

`assignLicense -> 403` **on `--apply`.** Reclamation needs a write role (e.g. License Administrator + `Directory.ReadWrite.All`), which is deliberately separate from the read tooling (R6). Assign the role just-in-time for the change window.

Projected saving is 0. No `--price-file` was supplied, so per-SKU prices are unknown. Provide a CSV of `skuId,monthly_price` (from your billing) to get a saving estimate; reclamation itself works without it.

A license was reclaimed from an account that was still needed. License assignment is reversible while the subscription is active — re-assign it in the admin center (or re-run with that user in scope of an add) within the change window. This is why subscriptions are cancelled only after Phase 2 export completes.

Phase 4 — identity & apps

An app registration you removed broke something. App-registration deletion is deliberately *not* automated by the tool — it is a manual, evidenced admin step precisely so it is deliberate. For service principals/accounts, always `--action disable` first and soak through the rollback window; re-enable a disabled object rather than restoring a deleted one.

A service principal keeps reappearing. It is backed by an app registration or a first-party Microsoft service and is being recreated. Disable the underlying app registration (evidenced), or confirm it is a required first-party SP that should be KEPT.

`--apply` **did nothing.** `--apply` acts only on ids/appIds present in `--scope-file`. Confirm the file lists the exact `id` (or `appId`) values from the inventory JSON, one per line.

Phase 5 — DNS & mail flow

Mail bounces or misroutes after a DNS change. Highest-impact failure. **Restore the pre-change zone snapshot immediately** (you took one before the change — Best Practices). Then re-plan: confirm MX points to Google (`aspmx.l.google.com`), SPF drops `spf.protection.outlook.com` and includes `_spf.google.com`, and DMARC still passes.

`map_dns_integration.py` **returns empty records for a domain.** Either the records genuinely don't exist, or the fallback resolver (`nslookup`) parsed poorly. Install `dnspython` (`pip install dnspython`) for reliable results, and cross-check with `dig` (Testing T5).

A Microsoft verification `MS=` **/** `msoid` **record still resolves.** Remove it only after the domain is removed from the tenant in Phase 6 (removing it too early can block domain removal). Sequence: clear routing records → remove domain from tenant → clear verification record.

Phase 6 — closure

`decommission_report.py --precheck` **returns** `FAIL`**.** It names the blocker. Typical causes: a gate has no evidence file (re-run that phase's tool to the evidence dir), completeness is not PASS, a domain is still verified/routing (finish Phase 5), a license is still assigned to a non-retained account (finish Phase 3), an app registration other than `Decommission Read – Source-Tenant – v1` remains (finish Phase 4), or an UNCLASSIFIED disposition item remains (Phase 2). Resolve, then re-run precheck until PASS before cancelling any subscription.

Domain won't remove from the tenant. Microsoft blocks removal while objects still reference the domain (users, groups, apps) or DNS verification records remain. Clear referencing objects and records first. Exact prerequisites are version-dependent — verify in your tenant.

"Where is the delete-tenant action?" Whether a tenant is deleted explicitly or lapses after all subscriptions are cancelled and users/licenses cleared is version-dependent — verify current tenant-deletion requirements in your tenant. Do not force anything; follow Microsoft's current documented path, and remember closure is irreversible past the deletion window.

Best Practices

Durable practices for retiring a Microsoft 365 tenant cleanly and auditably after a move to Google.

Sequence and gates

- **Never tear down before completeness passes.** The single most common way to lose data is to start

disposition or closure while a subset of users still depend on Microsoft 365. Gate 1 is a hard stop.

- **One direction, gated.** Completeness → disposition → licensing → identity → DNS → closure. Each

gate produces evidence; a later phase never starts until the prior gate is green.

- **Closure is last and irreversible.** Everything reversible (disable before delete, re-assign a

license, restore a zone) happens before the tenant enters Microsoft's deletion window. Treat the grace period as your final safety margin, not a rollback plan.

Safe-by-default automation

- **Read-only or** `--dry-run` **until** `--apply`**.** All five tools default to no-change mode. Review the

plan, get sign-off, then apply — pilot scope first.

- **Least privilege for verification.** The tooling app (`Decommission Read – Source-Tenant – v1`)

holds **read** scopes only. Write actions use just-in-time admin roles (R6), removed after the change window.

- **No secrets in code or repo.** Tenant/client IDs and the client secret come from the environment

(R3). The tools refuse to run if the variables are absent — never hardcode, never print, never commit credentials or exported service-account/app keys.

Data & legal discipline

- **Legal/records owns the determination; you own the method and the evidence.** Inventory and export

are technical; HOLD/RETAIN/DISPOSE is a legal decision. Get disposition sign-off in writing before any deletion.

- **Disable before delete.** For ambiguous identities and apps, disable and soak through the rollback

window before removal — a reversible step that catches "actually, that was still in use".

- **Preserve first, dispose second.** Export everything under hold/retention to the governed evidence

store *before* any DISPOSE action; record item, size, and hash so the export is verifiable.

Identity & app hygiene

- **Enumerate standing secrets.** Migrations leave app registrations with long-lived secrets/certs and

orphaned service principals — prime attack surface. Inventory them explicitly and remove what the Google move made obsolete.

- **Kill the tooling app last.** The read app is the final object removed, after it has produced the

closure evidence.

DNS & integration

- **Snapshot every zone before you touch it.** A pre-change export of the zone is your rollback for mail-flow records. Never edit MX/SPF/DKIM/DMARC blind.
- **One zone at a time, under change control.** Mail-flow records are high-impact; change one domain, validate externally (`dig`, a test message), then move on.
- **Re-point survivors to Google explicitly.** Confirm the Google target records (MX, SPF include, DKIM selector) in the Google Admin console before removing the Microsoft equivalents.

Evidence & auditability

- **Every gate leaves a file.** JSON/CSV inventories per phase, plus the final HTML closure report, archived in `Decommission Evidence – Source-Tenant – v1` with retention at least as long as the longest applicable obligation.
- **Name artifacts consistently.** Use the `<Purpose> – <Scope> – <Stage>` convention and reuse the exact names across steps, scripts, and evidence — an auditor should be able to follow one thread.

Vendor-accuracy discipline

- **Flag version/preview-dependent mechanics.** Microsoft's tenant-deletion prerequisites, Purview eDiscovery export navigation, and domain-removal rules change; where this matters, verify in your tenant rather than trusting a remembered path. Google-side re-point targets are confirmed in the Google Admin console at the time of change.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

- **Solution Overview** — the retirement model (completeness verification, data disposition vs retention/legal-hold, licensing wind-down, identity/app cleanup, DNS/integration cutover, formal closure), reference design, benefits, limitations, and the pilot-first deployment model.
- **Requirements Guide** — the solution-specific prerequisites (migration sign-off, least-privilege Graph read app, secret handling, retention/legal-hold inputs, evidence store, admin roles, DNS control, pilot scope), each with how to verify and how to implement if missing — building on the Google category's shared global requirements.
- **Implementation Guide** — the six phases with named artifacts, exact ordered steps, pass/fail gates, and rollback windows: Assess → Completeness → Disposition → Licensing → Identity → DNS → Closure.

• **Testing & Validation** — a validation procedure per gate, each with explicit PASS/FAIL signals, plus an end-to-end pilot acceptance test.

• **Troubleshooting** — common failure modes (auth, throttling, false completeness gaps, DNS/mail-flow breakage, closure blockers) and their resolutions.

• **Best Practices** — durable practices for a clean, auditable, safe-by-default exit.

• **Executive Summary** — the outcome, approach, and why it is safe and governed, for sponsors.

The 5 Python tools

All are **Python** using **Microsoft Graph** via `msal` (auth) + `requests` (calls); **read-only or** `--dry-run` **by default**, `--apply` required for any change; **no hardcoded credentials** (they read `M365DECOM_TENANT_ID` / `M365DECOM_CLIENT_ID` / `M365DECOM_CLIENT_SECRET` from the environment).

1. `verify_migration_completeness.py` — *assessment / discovery (read-only).* Reconciles each

in-scope user's mailbox item counts, last-activity, OneDrive usage, and group/shared-mailbox coverage against the migration baseline via Graph usage reports and directory reads, flagging residuals and post-cutover Microsoft-side activity. Feeds **Gate 1**.

1. `license_winddown_helper.py` — *configuration / provisioning helper (dry-run default).* Models

assigned SKUs per user, computes reclaimable vs retained seats and the projected monthly saving, and — only with `--apply` — removes license assignments (never cancels subscriptions) for a named scope. Feeds **Gate 3**.

1. `inventory_app_and_orphans.py` — *policy / guardrail + inventory.* Enumerates Entra app

registrations, enterprise apps (service principals), long-lived/expired credentials, and stale/guest/orphaned accounts; classifies them; and — with `--apply` — disables (default) or removes named objects. Also runs in `--mode data` to inventory data-bearing objects for disposition. Feeds **Gates 2 and 4**.

1. `map_dns_integration.py` — *scope / config mapper (read-only).* For each domain, resolves and

classifies MX, SPF, DKIM, DMARC, autodiscover, `enterpriseregistration`/`enterpriseenrollment`, Teams SRV, and Microsoft verification records as RE-POINT-TO-GOOGLE / REMOVE / KEEP, and lists Entra verified domains and app reply URLs as integration signals. Plans the Phase 5 cutover. Feeds **Gate 5**.

1. `decommission_report.py` — *validation / report generator.* Runs the pre-closure guardrail

`--precheck` (all prior gates evidenced, no routing domain, no assigned license on non-retained accounts, no residual apps, zero unresolved holds) and generates the final **HTML closure evidence pack** consolidating every gate's result and the subscription-cancellation dates. Feeds **Gate 6**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

decommission_report.py

inventory_app_and_orphans.py

license_winddown_helper.py

map_dns_integration.py

verify_migration_completeness.py